

Taller: Análisis de vulnerabilidades con el SIEM de Wazuh

Tabla de contenidos

- [Objetivos del taller](#)
- [Guía paso a paso](#)
- [Qué son los logs](#)
- [Arquitectura de Wazuh SIEM](#)
- [Agregar logs de aplicaciones \(Apache/Nginx y otros\)](#)
- [Crear y personalizar reglas de advertencia](#)
- [Reglas predefinidas: MITRE ATT&CK, NIST y CIS](#)
- [Pruebas y validación de reglas](#)
- [Ejercicios prácticos guiados](#)
- [Buenas prácticas y recomendaciones](#)

Objetivos del taller

- Comprender qué son los logs y su rol en un SIEM.
- Configurar agentes de Wazuh para recolectar logs de aplicaciones web (Apache/Nginx).
- Crear y personalizar reglas de advertencia para detectar comportamientos sospechosos.
- Aplicar pruebas para validar que las reglas funcionen correctamente.

Guía paso a paso

1. **Verificar el entorno Wazuh:** confirma que el Manager y el Dashboard están activos y accesibles.
2. **Registrar el agente:** instala y registra un Wazuh Agent en el servidor que genera los logs (Linux/Windows según tu plataforma).
3. **Configurar recolección de logs:** edita `ossec.conf` del agente para incluir los logs de Apache/Nginx u otras apps.
4. **Reiniciar el agente:** aplica cambios reiniciando el servicio del agente.

5. **Crear reglas:** añade reglas locales en `local_rules.xml` del Manager para detectar patrones relevantes.
6. **Probar reglas:** usa `wazuh-logtest` y genera tráfico real para validar detecciones.
7. **Ajustar y documentar:** calibra niveles, frecuencia y decoders; documenta decisiones y resultados.

Este taller evita una sección de "requisitos previos" y guía cada acción directamente en los apartados correspondientes.

Qué son los logs

Los logs son registros cronológicos de eventos generados por sistemas, aplicaciones y dispositivos. Permiten reconstruir acciones, diagnosticar problemas y detectar comportamientos anómalos.

Componentes habituales de un evento

- **Timestamp:** fecha y hora del evento.
- **Origen:** host, servicio o aplicación que lo generó.
- **Nivel:** severidad (info, warn, error, critical).
- **Mensaje:** descripción del evento, campos y contexto.

Los formatos más comunes incluyen *syslog*, líneas de texto estructurado, y *JSON*. La consistencia del formato facilita la decodificación y correlación.

Arquitectura de Wazuh SIEM

- **Agentes:** recolectan logs y eventos en cada host.
- **Manager:** recibe, decodifica y evalúa eventos contra reglas.
- **Indexador** (OpenSearch/Elasticsearch): almacena alertas y eventos.
- **Dashboard:** visualiza y explora detecciones.

Flujo: el agente envía eventos → el Manager aplica *decoders* para extraer campos → se evalúan *rules* → se generan alertas con niveles y etiquetas → se indexan y visualizan.

Agregar logs de aplicaciones (Apache/Nginx y otros)

Para recolectar logs de aplicaciones con Wazuh Agent, se define la ubicación y el formato en `ossec.conf` del agente.

1. **Editar** el archivo `ossec.conf` del agente.

2. **Agregar** bloques `<localfile>` con el `log_format` correcto y la `location` del archivo de log.
3. **Reiniciar** el agente para aplicar los cambios.
4. **Verificar** en el Dashboard que los eventos comienzan a llegar.

1) Apache HTTP Server

Ubicaciones comunes de access logs: `/var/log/apache2/access.log` (Debian/Ubuntu) o `/var/log/httpd/access_log` (RHEL/CentOS).

```
<localfile>
  <log_format>apache</log_format>
  <location>/var/log/apache2/access.log</location>
</localfile>

<localfile>
  <log_format>apache</log_format>
  <location>/var/log/apache2/error.log</location>
</localfile>
```

2) Nginx

Ubicación común: `/var/log/nginx/access.log` y `/var/log/nginx/error.log`.

```
<localfile>
  <log_format>nginx</log_format>
  <location>/var/log/nginx/access.log</location>
</localfile>

<localfile>
  <log_format>nginx</log_format>
  <location>/var/log/nginx/error.log</location>
</localfile>
```

3) Aplicaciones personalizadas

Si tu app escribe en texto o JSON, usa `syslog` o `json` y crea decoders y reglas propias:

```
<localfile>
  <log_format>syslog</log_format>
  <location>/var/log/myapp/app.log</location>
</localfile>
```

Tras modificar `ossec.conf` en el agente:

- Linux: `sudo systemctl restart wazuh-agent`
- Windows: reinicia el servicio "Wazuh Agent" desde Services o `Restart-Service wazuh-agent`.

Crear y personalizar reglas de advertencia

En Wazuh, los **decoders** extraen campos de los eventos, y las **rules** definen condiciones para generar alertas (niveles, grupos, tags). Las reglas locales se almacenan en el Manager: `/var/ossec/etc/rules/local_rules.xml`.

Decoders (si necesitas extraer campos de un formato propio)

Ubicación: `/var/ossec/etc/decoders/local_decoder.xml`. Ejemplo sencillo para una línea personalizada:

```
<decoder name="myapp-decoder">
  <prematch>myapp</prematch>
  <regex>^time=(\S+) ip=(\S+) level=(\w+) msg="(.*)"$</regex>
  <order>timestamp,srcip,level,msg</order>
</decoder>
```

Con este decoder, las reglas pueden usar campos como `srcip`, `level` o `msg`.

Reglas para Apache/Nginx (detección de rutas sensibles)

Ejemplo de reglas locales que detectan accesos a rutas sensibles y escalado por frecuencia:

```
<group name="web,sensitive-paths">
  <rule id="100100" level="7">
    <description>Acceso a rutas sensibles (.env, wp-admin, phpmyadmin)</description>
    <field name="url">(^|\w+)\.(env)|(^|\w+)wp-admin|(^|\w+)phpmyadmin</field>
```

```

    <options>no_full_log</options>
    <group>apache,nginx</group>
    <mitre>TA0009,TA0043</mitre>
</rule>

<rule id="100101" level="10" frequency="5" timeframe="120">
    <description>Múltiples accesos a rutas sensibles desde la misma IP</description>
    <if_matched_sid>100100</if_matched_sid>
    <same_source/>
    <group>apache,nginx</group>
</rule>
</group>

```

Regla de errores 404 repetidos (posible exploración)

```

<group name="web,bruteforce-or-scan">
    <rule id="100110" level="6">
        <description>HTTP 404 detectado en web server</description>
        <field name="status">404</field>
        <group>apache,nginx</group>
    </rule>

    <rule id="100111" level="9" frequency="20" timeframe="300">
        <description>Muchos 404 en poco tiempo desde misma IP</description>
        <if_matched_sid>100110</if_matched_sid>
        <same_source/>
        <group>apache,nginx</group>
    </rule>
</group>

```

Pasos para crear reglas:

1. En el Manager, crear/editar `/var/ossec/etc/rules/local_rules.xml`.
2. Agregar el bloque `<group>...</group>` con reglas y guardar.

```
3. Reiniciar el Manager: sudo systemctl restart wazuh-manager o  
/var/ossec/bin/wazuh-control restart.
```

Reglas predefinidas: MITRE ATT&CK, NIST y CIS

Wazuh incluye un amplio conjunto de reglas preconfiguradas que referencian marcos de seguridad reconocidos: **MITRE ATT&CK**, **NIST 800-53** y **CIS Benchmarks** (entre otros como PCI DSS, HIPAA, GDPR). Estas referencias ayudan a clasificar y reportar hallazgos según estándares internacionales.

MITRE ATT&CK

MITRE ATT&CK es un marco que describe tácticas y técnicas usadas por adversarios. Componentes clave:

- **Táctica** (TAxxxx): objetivo del atacante, p.ej. *Defense Evasion* (TA0005), *Persistence* (TA0003), *Credential Access* (TA0006).
- **Técnica** (Txxxx) y **Subtécnica** (Txxxx.x): método concreto, p.ej. *Masquerading* (T1036), *Obfuscated/Compressed Files* (T1027).
- **Fuentes de datos**: eventos de proceso, archivo, red, registro; útiles para detección.

Defense Evasion (TA0005) agrupa técnicas para evitar detección: borrar/alterar logs, desactivar herramientas de seguridad, ofuscar binarios, cambiar nombres (masquerading). Las reglas de Wazuh etiquetadas con TA0005 señalan comportamientos compatibles con estas técnicas.

```
<rule id="100120" level="8">  
  <description>Intentos de login fallidos reiterados</description>  
  <group>authentication,syslog</group>  
  <mitre>TA0006,T1110</mitre> <!-- Credential Access / Brute Force -->  
</rule>
```

En el Dashboard puedes filtrar por MITRE (p.ej., buscar TA0005 o por texto Defense Evasion) para analizar tácticas específicas.

NIST SP 800-53

NIST 800-53 define controles de seguridad y privacidad para sistemas de información. Componentes clave:

- **Familias de control:** *AC* (Control de acceso), *AU* (Auditoría), *SI* (Monitoreo del sistema), etc.
- **Controles:** identificadores como AC-7 (Límites a intentos fallidos), AU-6 (Revisión de auditoría), SI-4 (Monitoreo continuo).

```
<rule id="100121" level="9">
  <description>Excesivos intentos de autenticación fallidos</description>
  <group>authentication</group>
  <compliance>
    <nist_800_53>AC-7</nist_800_53>
  </compliance>
</rule>
```

Estas etiquetas permiten generar reportes de cumplimiento y paneles filtrando por controles NIST relevantes.

CIS Benchmarks

Los CIS Benchmarks son guías de endurecimiento (hardening) por plataforma: Linux, Windows, Kubernetes, etc. Componentes clave:

- **Benchmark y Sección:** recomendación aplicable (p.ej., CIS Debian 1.1.2: registrar intentos de login).
- **Severidad:** impacto y prioridad de la recomendación.

```
<rule id="100122" level="7">
  <description>Configuración de auditoría acorde a CIS</description>
  <group>configuration,compliance</group>
  <compliance>
    <cis>1.1.2</cis>
  </compliance>
</rule>
```

¿Dónde ver estas referencias en Wazuh?

- Reglas predefinidas: `/var/ossec/ruleset/rules/` y `/var/ossec/etc/rules/`.
- Busca etiquetas `<mitre>` y `<compliance>` dentro de las reglas.

- En el Dashboard, filtra por MITRE/NIST/CIS para ver correlaciones y cumplimiento.

Cómo añadir estas referencias a tus reglas locales

Al crear reglas en `local_rules.xml`, añade etiquetas de mapeo para clasificación y reporting:

```
<rule id="100130" level="8">
  <description>Acceso a rutas sensibles con patrón sospechoso</description>
  <group>apache,nginx,web</group>
  <field name="url">(^|\|\/)\.(env)|(^|\|\/)wp-admin|^|\|\/)phpmyadmin</field>
  <mitre>TA0043,T1595</mitre> <!-- Reconnaissance / Active Scanning -->
  <compliance>
    <nist_800_53>SI-4</nist_800_53>
    <cis>5.1.1</cis>
  </compliance>
</rule>
```

Estas referencias no generan la alerta por sí mismas, pero enriquecen el contexto y facilitan auditoría y análisis táctico.

Pruebas y validación de reglas

Usa `wazuh-logtest` para validar reglas fuera de línea y luego genera eventos reales para confirmar la detección en el Dashboard.

1) Pruebas con wazuh-logtest

Ejecuta en el Manager:

```
sudo /var/ossec/bin/wazuh-logtest
```

Luego pega una línea de log de ejemplo de Apache/Nginx que debería coincidir con tus reglas. Ejemplo (petición a `/.env`):

```
127.0.0.1 - - [12/Oct/2025:12:34:56 +0000] "GET /.env HTTP/1.1" 200 123
```


El resultado mostrará qué decoders y reglas han coincidido (incluyendo los IDs 100100/100101 si aplica).

2) Generar eventos reales

- Realiza peticiones con curl a rutas sensibles: `curl -I http://tu-host/.env`.
- Genera 404 repetidos en poco tiempo para probar la regla de frecuencia.
- Verifica en Wazuh Dashboard las alertas y niveles (7/10 o 6/9).

3) Validaciones adicionales

- Confirma que los campos (url, status, srcip) se extraen correctamente; si no, ajusta decoders.
- Comprueba que same_source agrupa por IP; ajusta si tu campo difiere.
- Revisa que los niveles (level) y grupos sean coherentes con tus políticas.

Ejercicios prácticos guiados

Ejercicio A: Recolección de logs de Apache

1. En el agente del servidor web, agrega los bloques <localfile> para access.log y error.log.
2. Reinicia el agente y verifica la conectividad con el Manager.
3. En Dashboard, confirma que aparecen eventos de Apache.

Ejercicio B: Regla de rutas sensibles

1. Crea las reglas 100100 y 100101 en local_rules.xml.
2. Prueba con wazuh-logtest usando una línea con /.env.
3. Genera tráfico real y verifica alertas en Dashboard.

Ejercicio C: 404 de exploración

1. Implementa las reglas 100110 y 100111.
2. Usa un script para intentar rutas inexistentes y provocar 404.
3. Comprueba la alerta de frecuencia y ajusta parámetros si es necesario.

Buenas prácticas y recomendaciones

- **Versiona** tus reglas y decoders; documenta cambios y motivos.

- Usa **grupos y etiquetas** coherentes para facilitar búsquedas y paneles.
 - Evita **falsos positivos** afinando patrones y añadiendo condiciones.
 - Aplica **parámetros de frecuencia y timeframe** con valores realistas.
 - Monitorea el **rendimiento**: demasiadas reglas de alta frecuencia pueden impactar.
-